

KA4 - Guia de Construção de Software

Sistema de Gestão de Consultório Médico

Arquitetura de Software

3 de junho de 2026

1 Introdução e Contexto Tecnológico

Em alinhamento com a definição arquitetural (KA2) que estabelece um **Monolito Modular** baseado em Client-Server, este guia (KA4) define as práticas de construção de software. Para garantir aplicabilidade técnica, este guia é direcionado para a stack tecnológica adotada: **TypeScript**, utilizando **React (SPA)** no Frontend, **Node.js (NestJS/Express)** na API Backend e **PostgreSQL** como Banco de Dados Relacional. Os princípios de *Clean Code* (Martin, 2008) e **SOLID** formam a base destas diretrizes, garantindo manutenibilidade e isolamento entre os módulos lógicos (Autenticação, Pacientes, Prontuário, etc.).

2 Guia de Padrões de Codificação

2.1 Convenções de Nomenclatura

Seguindo o princípio de *Clean Code* de que nomes devem revelar intenção, adotamos o padrão idiomático do ecossistema TypeScript/JavaScript:

- **Classes, Interfaces e Types:** PascalCase (ex: PatientController, MedicalRecord).
- **Variáveis, Funções e Métodos:** camelCase (ex: getPatientById, isAuthorized).
- **Constantes Globais:** UPPER_SNAKE_CASE (ex: MAX_LOGIN_ATTEMPTS).
- **Arquivos e Diretórios:** kebab-case (ex: patient-repository.ts, medical-records/).
- **Banco de Dados (Tabelas e Colunas):** snake_case (ex: patient_records, created_at).

2.2 Organização do Código-Fonte (Monolito Modular)

Para suportar o **Princípio da Responsabilidade Única (SRP)** do SOLID e manter os limites lógicos do monolito modular, a estrutura de pastas do backend **deve** ser orientada a domínio (Módulos), e não a camadas técnicas globais.

```
src/  
|-- modules/  
|   |-- patients/           # Domínio de Pacientes  
|   |   |-- controllers/    # Camada de Apresentação (Rotas HTTP)  
|   |   |-- services/       # Camada de Regras de Negócio  
|   |   |-- repositories/   # Camada de Acesso a Dados (DB)  
|   |   |-- patient.entity.ts # Modelo de Domínio  
|   |-- medical-records/    # Domínio de Prontuários (Isolado)  
|-- shared/                 # Código comum (Utils, Middlewares de Auth)
```

2.3 Tratamento de Exceções

Não retorne status HTTP diretamente de serviços de domínio. Utilize exceções customizadas para erros de negócio. O *Controller* ou um *Global Exception Filter* deve traduzir a exceção para o erro HTTP adequado.

X Incorreto (Acoplamento do Domínio com HTTP e vazamento de stack):

```
1 async function getProntuario(id: string, res: Response) {
2   try {
3     const prontuario = await db.find(id);
4     if (!prontuario) return res.status(404).send('Nao encontrado');
5   } catch (error) {
6     // VAZAMENTO DE DADOS: Retornar error.message expoe infraestrutura
7     return res.status(500).json({ error: error.message });
8   }
9 }
```

✓ Correto (Separação de responsabilidades e segurança):

```
1 // domain/exceptions/not-found.exception.ts
2 export class RecordNotFoundException extends Error { ... }
3
4 // services/medical-record.service.ts
5 async function getProntuario(id: string): Promise<MedicalRecord> {
6   const prontuario = await repository.findById(id);
7   if (!prontuario) throw new RecordNotFoundException(`Prontuario ${id} nao encontrado.`);
8   return prontuario;
9 }
```

2.4 Regras de Comentários e Documentação Inline

Conforme *Clean Code*, comentários não devem compensar código ruim. O código deve ser autoexplicativo através de bons nomes de variáveis e funções.

- Evite comentários óbvios (ex: `// incrementa i`).
- Utilize **JSDoc** (`/** ... */`) exclusivamente para documentar APIs públicas de interfaces, contratos de serviços de domínio e parâmetros complexos.
- Comentários de bloco devem explicar o *"Porquê"* (decisão de negócio), nunca o *"O quê"* (o código já diz isso).

2.5 Complexidade Ciclômática Máxima

A Complexidade Ciclômática mede o número de caminhos lineares independentes no código.

- **Limite Tolerado:** A complexidade ciclômática máxima tolerada por função/método é **10** (medida via SonarQube/ESLint).
- Justificativa (Clean Code): Funções devem ser pequenas e fazer apenas uma coisa (SRP). Se um método possui múltiplos `if/else` ou `switch` extensos, ele deve ser refatorado utilizando polimorfismo, *Strategy Pattern* ou extração de métodos auxiliares.

3 Guia de Práticas de Construção Segura

Devido à criticidade do sistema (dados de saúde), a segurança e a aderência à **LGPD** são inegociáveis.

3.1 Manipulação de Dados Sensíveis e Logs (LGPD)

É absolutamente **proibido registrar Dados Pessoais Identificáveis (PII) ou Informações de Saúde de Saúde Protegidas (PHI) em logs textuais ou ferramentas de monitoramento sem mascaramento.**

- **Regra de Ouro (LGPD):** Campos como nome do paciente, CPF, detalhes do diagnóstico e endereço de e-mail **NUNCA** devem aparecer em comandos `console.log()`, Datadog, Sentry ou Kibana.
- **Mascaramento (Data Masking):** Utilize bibliotecas de *logger* (ex: Winston ou Pino no Node.js) configuradas com *redaction* para ofuscar chaves sensíveis automaticamente.
- **Criptografia em Repouso:** Arquivos anexos (ex: PDFs de exames) devem ser encriptados no Storage (AES-256). Senhas de usuários devem ser haseadas com **Argon2** ou **Bcrypt** (com *salt* gerado aleatoriamente).

✗ Incorreto (Violação LGPD):

```
1 logger.info(`Atualizando diagnostico para paciente: ${paciente.nome}, CPF: ${paciente.cpf}, Doenca: ${diagnostico}`);
```

✓ Correto (Seguro e Adimplente com a LGPD):

```
1 // O UUID é um identificador opaco, seguro para logs.
2 logger.info(`Prontuario atualizado com sucesso. PatientID: ${paciente.id}`);
```

3.2 Validação e Sanitização de Entradas

- **Nunca confie no Frontend:** Toda validação feita no React (SPA) deve ser obrigatoriamente replicada na API Backend.
- **Whitelist, não Blacklist:** Valide os dados contra um esquema rígido esperado. Recomenda-se o uso de bibliotecas como **Zod** ou **class-validator** no DTO (Data Transfer Object).
- Evite injeção de SQL utilizando estritamente *Prepared Statements* ou um ORM seguro (ex: Prisma, TypeORM).

3.3 Uso de Bibliotecas Externas (Third-Party)

- A execução do comando `npm audit` deve fazer parte da esteira CI/CD (Pipeline).
- **Licenças permitidas:** MIT, Apache 2.0, ISC.
- **Proibido:** Bibliotecas com licenças restritivas (GPLv3) sem aprovação expressa do Tech Lead, ou dependências não atualizadas há mais de 12 meses (Risco de vulnerabilidade).

4 Modelo de Revisão de Código (Code Review Checklist)

Todo *Pull Request* (PR) deve ser aprovado por pelo menos um desenvolvedor sênior utilizando o checklist abaixo antes do *merge* na branch *main*.

Status	Critério Técnico de Aceitação do Pull Request (PR)
☐	Arquitetura: O código respeita os limites do Monólito Modular? (ex: O módulo de <i>Agendamento</i> não acessa diretamente o banco de dados do <i>Prontuário</i> , comunicando-se apenas via interfaces de serviço?).
☐	Clean Code / SOLID: As funções possuem uma única responsabilidade (SRP)? As classes dependem de abstrações em vez de implementações concretas (DIP)? A complexidade ciclomática está abaixo de 10?
☐	Privacidade e LGPD: Foi verificado se NENHUM dado pessoal (CPF, nome, doença) está sendo inserido em logs ou exceptions? O <i>logger</i> obfusca adequadamente objetos de Payload?
☐	Segurança (Autorização): Os novos <i>endpoints</i> da API possuem o decorador/middleware de verificação RBAC (Role-Based Access Control)? (ex: Apenas perfis de Médico podem alterar prontuários).
☐	Sanitização: Todas as novas rotas expostas possuem validação de DTO (Zod/Class-Validator) ignorando propriedades não mapeadas (<i>whitelist: true</i>)?
☐	Testes Automáticos: Foram adicionados testes unitários para a nova regra de negócio (cobertura exigida de 80% do domínio)?